Logistic regression

Michel Jean Joseph Donnet

May 27, 2024

Contents

1 Introduction

There are many method to classify elements thanks to features given.

We are looking at the regression method, and study the performances of this method

2 Methodology

- X: a set of features ($n \times d$)
- y: a set of labels for the given features ($n \times h$)
- C: the set of all the classes

2.1 Binary case

In this case, $h = 1$.

We consider that we have only two classes $c_1, c_2 \in C$.

We want to define what is the probability of $p(y|X)$.

To do that, we want to define what is the probability for just one instance:

$$p(y_i = c_1|x_i)$$

As we have 2 classes, we can consider that this probability follow a Bernoulli law. So $y_i = 1$ if $y_i = c_1$ and $y_i = 0$ if $y_i \neq c_1$.

The probability to have 1 is given by a function that depends on x_i. So we have:

$$p(y_i = 1|x_i) = g(x_i) \tag{1}$$

with $g(x_i)$ which give us a scalar from the vector of features x_i.

So it means that we have:

$$p(y_i|x_i) = \begin{cases} p(y_i = 1|x_i) \text{ if } y_i = c_1 \\ 1 - p(y_i = 1|x_i) \text{ if } y_i \neq c_1 \end{cases} \tag{2}$$

We can write the equation 2 like this:

$$p(y_i|x_i) = p(y_i = 1|x_i)^{y_i}(1 - p(y_i = 1|x_i))^{1-y_i} \tag{3}$$

But we want to know what is the function $g(x_i)$.

According to Bayes theorem, we have:

$$
\begin{aligned}
p(y_i = 1|x_i) &= \frac{p(x_i|y_i = 1)p(y_i = 1)}{p(x_i)} \\
&= \frac{p(x_i|y_i = 1)p(y_i = 1)}{p(x_i|y_i = 1)p(y_i = 1) + p(x_i|y_i = 0)p(y_i = 0)} \\
&= \frac{\frac{p(x_i|y_i=1)p(y_i=1)}{p(x_i|y_i=0)p(y_i=0)}}{\frac{p(x_i|y_i=1)p(y_i=1)}{p(x_i|y_i=0)p(y_i=0)} + 1} \\
&= \frac{\frac{p(x_i|y_i=1)p(y_i=1)p(x_i)}{p(x_i|y_i=0)p(y_i=0)p(x_i)}}{\frac{p(x_i|y_i=1)p(y_i=1)p(x_i)}{p(x_i|y_i=0)p(y_i=0)p(x_i)} + 1} \\
&= \frac{\frac{p(y_i=1|x_i)}{p(y_i=0|x_i)}}{\frac{p(y_i=1|x_i)}{p(y_i=0|x_i)} + 1}
\end{aligned}
\tag{4}
$$

In probability, if q is the probability of an event, so the probability that this event is happening is given by $\frac{q}{1-q}$. This ratio is called the odds.

In our case, as $p(y_i = 1|x_i)$ is the probability that the event is happening, and $p(y_i = 0|x_i)$ is the probability that the event is not happening, we have the odds defined like this:

$$
Odds = \frac{p(y_i = 1|x_i)}{1 - p(y_i = 1|x_i)} \tag{5}
$$

In logistic regression, we want that the logarithm of the odds is linear. So according to equation 5, we want that:

$$
\begin{aligned}
& log\left(\frac{p(y_i = 1|x_i)}{p(y_i = 0|x_i)}\right) = w_0 + w_1 \cdot x_{i1} + \cdots + w_d \cdot x_{id} \\
\Leftrightarrow \quad & log\left(\frac{p(y_i = 1|x_i)}{p(y_i = 0|x_i)}\right) = w_0 + x_i w \\
\Leftrightarrow \quad & log\left(\frac{p(y_i = 1|x_i)}{p(y_i = 0|x_i)}\right) = x_i w \qquad \text{if } x = \begin{bmatrix} 1 & x_{i1} & \cdots & x_{id} \end{bmatrix} \\
& \qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad \text{and } w^T = \begin{bmatrix} w_0 & \cdots & w_d \end{bmatrix} \\
\Leftrightarrow \quad & \frac{p(y_i = 1|x_i)}{p(y_i = 0|x_i)} = e^{x_i w}
\end{aligned}
\tag{6}
$$

So according to the result of the equation 6, we can rewrite the equation 4 like this:

$$
\begin{aligned}
p(y_i = 1|x_i) &= \frac{\frac{p(y_i=1|x_i)}{p(y_i=0|x_i)}}{\frac{p(y_i=1|x_i)}{p(y_i=0|x_i)} + 1} \\
&= \frac{e^{x_i w}}{e^{x_i w} + 1} \\
&= \frac{1}{1 + e^{-x_i w}} \\
&= \sigma(x_i w) \qquad\qquad \text{with } \sigma(z) = \frac{1}{1 + e^{-z}}
\end{aligned} \tag{7}
$$

σ in 7 is called the sigmoïd function. (As e^{-x} is always negative or equal to zero, σ is always between 0 and 1)

So we can write the equation 3 like this:

$$
\begin{aligned}
p(y_i|x_i) &= p(y_i = 1|x_i)^{y_i}(1 - p(y_i = 1|x_i))^{1-y_i} \\
&= \sigma(x_i w)^{y_i}(1 - \sigma(x_i w))^{1-y_i}
\end{aligned} \tag{8}
$$

2.1.1 Cost function

Now, we want to define a cost function to optimize the parameter w.

Note that $p(y_i|x_i)$ is called the likelihood.

In fact, we want to maximise the probability of $p(y_i = 1|x_i)$ when $y_i = 1$ and maximise the probability of $(1 - p(y_i = 1|x_i))$ when $y_i \neq 1$.

So we want to maximize the likelihood, which is given by equation 8. And we want to maximize the probability for all instances and not only for instance i.

So we want to maximize:

$$
\max \arg_w \frac{1}{n} \sum_i^n p(y_i|x_i) \tag{9}
$$

The $\frac{1}{n}$ allow us to normalize our datas, to have an answer in the space definition of $p(y_i|x_i)$.

We want to use the gradient descent. So we want to have something to minimise. As we have to maximize the equation 8, we only have to invert this equation to have a minimisation problem.

We can easily see that the likelihood is always non zeros: we know that the exponential function is strictly positive, and according to equation 7, we have:

$$
\begin{aligned}
& 0 && < e^{x_i w} < 1 + e^{x_i w} \\
\Leftrightarrow \quad & 0 && < \frac{e^{x_i w}}{1 + e^{x_i w}} < 1 \\
\Leftrightarrow \quad & 0 && < sigma(x_i w) < 1 \\
\Leftrightarrow \quad & 0 && < p(y_i|x_i) < 1
\end{aligned}
$$

because else, it means that all the elements are in one class, so we don't have two classes and we don't need to make some classification.

So we have according to equation 9:

$$
\begin{aligned}
\max \arg_w \frac{1}{n} \sum_i^n p(y_i|x_i) &= \min \arg_w \frac{1}{n} \sum_i^n \frac{1}{p(y_i|x_i)} \\
&= \min \arg_w \frac{1}{n} \sum_i^n \frac{1}{\sigma(x_i w)^{y_i} (1 - \sigma(x_i w))^{1-y_i}} \\
&= \min \arg_w \frac{1}{n} \; log \left(\sum_i^n \frac{1}{\sigma(x_i w)^{y_i} (1 - \sigma(x_i w))^{1-y_i}} \right) \\
&= \min \arg_w \frac{1}{n} \sum_i^n \left(-log(\sigma(x_i w)^{y_i} (1 - \sigma(x_i w))^{1-y_i}) \right) \\
&= \min \arg_w \frac{1}{n} \sum_i^n \left(-y_i log(\sigma(x_i w)) - (1 - y_i) log(1 - \sigma(x_i w)) \right)
\end{aligned}
\tag{10}
$$

In the equation 10, we can apply log to the equation because the log function is always positive and increasing, and since we have said that $p(y_i|x_i)$ is always positive.

So we have inverted the likelihood, and then we have apply log function to result. As the log of the invert is equal to negative log (example: $log\left(\frac{1}{x}\right) = -log(x)$), we call the cost function the negative log likelihood function.

As y has a size of $n \times h$, X a size of $n \times d$ and w a size of $d \times h$, we can write the cost function with matrix product:

$$
\begin{aligned}
& \min \arg_w \frac{1}{n} \sum_i^n \left(-log(\sigma(x_i w)) y_i - log(1 - \sigma(x_i w))(1 - y_i) \right) \\
& = \min \arg_w \frac{1}{n} \left(-y^T log(\sigma(Xw)) - (1 - y^T) log(1 - \sigma(Xw)) \right) \\
& = NLL(X, y)
\end{aligned}
\tag{11}
$$

2.1.2 Gradient

Now we have to compute the derivate of the cost function 11.

First, we want to find the derivate of the sigmoid function.

We have:

$$\begin{aligned}
\frac{d\sigma(z)}{dz} &= ((1+e^{-z})^{-1}) \\
&= -1 \cdot \left(-e^{-z}\right) \cdot (1+e^{-z})^{-2} \\
&= \frac{e^{-z}}{(1+e^{-z})^2} \\
&= \frac{1}{1+e^{-z}} \cdot \frac{e^{-z}}{1+e^{-z}} \\
&= \sigma(z)\frac{e^{-z}}{1+e^{-z}} \\
&= \sigma(z)\frac{1+e^{-z}-1}{1+e^{-z}} \\
&= \sigma(z)(\frac{1+e^{-z}}{1+e^{-z}} - \frac{1}{1+e^{-z}}) \\
&= \sigma(z)(1 - \frac{1}{1+e^{-z}}) \\
&= \sigma(z)(1-\sigma(z))
\end{aligned} \tag{12}$$

So according to the chain rule, we have:

$$\begin{aligned}
\frac{\partial}{\partial w_k} NLL(X,y) &= \frac{1}{n}\left(\frac{\partial}{\partial w_k}\left(-y^T log(\sigma(Xw)) - (1-y^T) log(1-\sigma(Xw))\right)\right) \\
&= \frac{1}{n}\left(-y^T \frac{\partial}{\partial w_k} log(\sigma(Xw)) - (1-y^T)\frac{\partial}{\partial w_k} log(1-\sigma(Xw))\right) \\
&= \frac{1}{n}\left(-y^T \frac{\partial log(\sigma(Xw))}{\partial \sigma}\frac{\partial \sigma}{\partial Xw}\frac{\partial Xw}{\partial w_k}\right. \\
&\quad \left. -(1-y^T)\frac{\partial log(1-\sigma(Xw))}{\partial(1-\sigma(Xw))}\frac{\partial(1-\sigma(Xw))}{\partial \sigma}\frac{\partial \sigma(Xw)}{\partial Xw}\frac{\partial Xw}{\partial w_k}\right) \\
&= \frac{1}{n}\left(-y^T \frac{1}{\sigma(Xw)}\sigma(Xw)(1-\sigma(Xw))X_{:,k}\right. \\
&\quad \left. -(1-y^T)\frac{1}{1-\sigma(Xw)}(-1)\sigma(Xw)(1-\sigma(Xw))X_{:,k}\right) \\
&= \frac{1}{n}\left(((1-y^T)\sigma(Xw) - y^T(1-\sigma(Xw)))X_{:,k}\right) \\
&= \frac{1}{n}\left((\sigma(Xw) - y^T\sigma(Xw) - y^T 1_{n,1} + y^T\sigma(Xw))X_{:,k}\right) \\
&= \frac{1}{n}\left(\left(\sigma(Xw) - \sum_i^n y_i\right)X_{:,k}\right)
\end{aligned} \tag{13}$$

So the gradient of the cost function give us:

$$\nabla NLL(X,y) = \begin{bmatrix} \frac{\partial}{\partial w_0} NLL(X,y) \\ \vdots \\ \frac{\partial}{\partial w_d} NLL(X,y) \end{bmatrix}$$

Now, we have to make a gradient descent:

$$w^{i+1} = w^i - \eta \nabla NLL(X,y) \tag{14}$$

with η the learning rate.

2.2 Multinomial case

Now, we have h classes.

If $y_i = class_k$, we define $\tilde{y}_i$ a vector of size $1 \times h$ like this:

$$\tilde{y}_{ij} = \begin{cases} 1 \text{ if} j = k \\ 0 \text{ else} \end{cases} \tag{15}$$

So we have for instance $\tilde{y_i} = \begin{bmatrix} 0 & 0 & 1 & 0 \end{bmatrix}$ if $h = 4$.

This class representation is called one-hot encoding.

Now, we note y_i the result of the one-hot encoding of y_i: $y_i = \tilde{y_i}$. So y is of size $n \times h$.

If we remember the equation 7, we have (binary case):

$$p(y_i = 1|x_i) = \frac{e^{x_i w}}{e^{x_i w} + 1} \tag{16}$$

We want to generalize the sigmoid function to more than two variables to have $p(c|x_i)$ the probability of class c in one-hot representation according to the features x_i.

Now, we suppose that we have

$$W = \begin{bmatrix} w_1 & \cdots & w_h \end{bmatrix} \tag{17}$$

with each parameter w_k defined specially for the class k. (Note that the size of w_k is $d \times 1$, so the size of W is $d \times h$)

In binary case, we can define $W = \begin{bmatrix} w_1 & w_2 \end{bmatrix}$, with w_1 a column vector of 0.

So we can write the equation 16 like this:

$$\begin{aligned} p(y_i = 1|x_i) &= \frac{e^{x_i w_2}}{e^{x_i w_2} + e^0} \\ &= \frac{e^{x_i w_2}}{e^{x_i w_2} + e^{x_i w_1}} \\ &= \frac{e^{x_i w_2}}{\sum_{k=1}^{2} e^{x_i w_k}} \\ &= \frac{e^{x_i w_2}}{\sum_{k=1}^{h} e^{x_i w_k}} \end{aligned} \tag{18}$$

So if we have h classes, we can define a function like this:

$$\begin{aligned} p(y_{ik} = 1|x_i) &= \frac{e^{x_i w_k}}{\sum_{j=1}^{h} e^{x_i w_j}} \\ &= softmax(W, x_i, k) \end{aligned} \tag{19}$$

And then, we can write this function in matricial form:

$$
\begin{aligned}
p(y_i|x_i) &= \begin{bmatrix} p(y_{i1}=1|x_i) & \cdots & p(y_{ih}=1|x_i) \end{bmatrix} \\
&= \begin{bmatrix} \frac{e^{x_i w_1}}{\sum_{j=1}^{h} e^{x_i w_j}} & \cdots & \frac{e^{x_i w_h}}{\sum_{j=1}^{h} e^{x_i w_j}} \end{bmatrix} \\
&= \frac{e^{x_i W}}{\sum_{j=1}^{h} e^{x_i w_j}} \\
&= softmax(x_i W)
\end{aligned}
\tag{20}
$$

We can easily see that $softmax$ is between 0 and 1: As the exponential function is always positive, we have that:

$$
\begin{aligned}
& 0 && < e^{x_i w_k} < e^{x_i w_k} + \sum_{j \neq k}^{h} e^{x_i w_j} \\
\Leftrightarrow \quad & 0 && < e^{x_i w_k} < \sum_{j=1}^{h} e^{x_i w_j} \\
\Leftrightarrow \quad & 0 && < e^{x_i w_k} < \sum_{j=1}^{h} e^{x_i w_j} \\
\Leftrightarrow \quad & 0 && < \frac{e^{x_i w_k}}{\sum_{j=1}^{h} e^{x_i w_j}} < 1 \\
\Leftrightarrow \quad & 0 && < softmax(W, x_i, k) < 1
\end{aligned}
\tag{21}
$$

and we can also see that $p(y_i|x_i)$ is a distribution of probability because we have:

$$
\begin{aligned}
\sum_{k}^{h} p(y_{ik}|x_i) &= \sum_{k}^{h} \frac{e^{x_i w_k}}{\sum_{j}^{h} e^{x_i w_j}} \\
&= \frac{\sum_{k}^{h} e^{x_i w_k}}{\sum_{j}^{h} e^{x_i w_j}} \\
&= \frac{\sum_{j}^{h} e^{x_i w_j}}{\sum_{j}^{h} e^{x_i w_j}} \\
&= 1
\end{aligned}
\tag{22}
$$

2.2.1 Cost function

Now we want to define a cost function.

We remember that we have X, the training features and y the labels corresponding to the features. We define W and we want to optimize this parameter.

We have:

Matrix	Size
X	$n \times d$
y	$n \times h$
W	$d \times h$

For all the instances, according to the result of the equation 20 and to the set of features X, the predicted values are given by:

$$p(c|X) = softmax(XW) \tag{23}$$

We can compute the cross-entropy between our predictions and the real values. The cross-entropy give us the difference between two distributions of probability. If we have two distributions P and Q, the cross-entropy is given by $H(P,Q) = -\frac{1}{m}\sum_x^m P(x)logQ(x)$

As the labels y are on one-hot form, we can considerate that y give us a distribution of probability because, due to the definition of y_i on one-hot encoding form, we have for each instance i the result below: $\sum_k y_{ik} = 1$. So for the labels y on one-hot form and for our predictions $p(c|X)$, we have the cross entropy which give us:

$$\begin{aligned} H(y, p(c|X)) &= -\frac{1}{n}\sum_i^n y_i \odot log(p(c|x_i)) \\ &= -\frac{1}{n}\sum_i^n log(softmax(x_i W)) \odot y \end{aligned} \tag{24}$$

With $\odot$ the Hadamard product.

As the cross-entropy is the difference between two distributions, we want to minimise this cross-entropy because we want that our predictions are equal to the real values y.

So we want to find an optimal W that minimise equation 24.

As we want to use a gradient descent to find the optimal W, we are searching for the gradient of the cross-entropy 24.

2.2.2 Gradient

First, we want to compute the partial derivate $\frac{\partial}{\partial w_k}$ of the cross-entropy:

$$\begin{aligned}
\frac{\partial}{\partial w_k} H(y, p(c|X)) &= -\frac{1}{n}\sum_i^n \frac{\partial}{\partial w_k} log(softmax(x_i W)) \odot y_i \\
&= -\frac{1}{n}\sum_i^n \frac{\partial}{\partial w_k} log(softmax(W, x_i, j)) && \text{with } y_{ij} = 1 \\
&= -\frac{1}{n}\sum_i^n \frac{\partial}{\partial w_k} log\left(\frac{e^{x_i w_j}}{\sum_{l=1}^h e^{x_i w_l}}\right) \\
&= -\frac{1}{n}\sum_i^n \frac{\partial}{\partial w_k} \left(x_i w_j - log\left(\sum_{l=1}^h e^{x_i w_l}\right)\right) \\
&= \frac{1}{n}\sum_i^n \left(\frac{\partial log\left(\sum_{l=1}^h e^{x_i w_l}\right)}{\partial \sum_{l=1}^h e^{x_i w_l}} \frac{\partial \sum_{l=1}^h e^{x_i w_l}}{\partial w_k} - \frac{\partial}{\partial w_k} x_i w_j\right) \\
&= \frac{1}{n}\sum_i^n \left(\frac{e^{x_i w_k} x_i^T}{\sum_{l=1}^h e^{x_i w_l}} - y_{ik} x_i^T\right) \\
&= \frac{1}{n}\sum_i^n \left(softmax(W, x_i, k) - y_{ik}\right) x_i^T
\end{aligned} \tag{25}$$

So the gradient of the cross-entropy is given by:

$$\begin{aligned}
\nabla_W H(y, p(c|X)) &= \left[\frac{\partial}{\partial w_1} H(y, p(c|X)) \quad \cdots \quad \frac{\partial}{\partial w_h} H(y, p(c|X))\right] \\
&= \left[\frac{1}{n}\sum_i^n \left(softmax(W, x_i, 1) - y_{i1}\right) x_i^T \quad \cdots \quad \frac{1}{n}\sum_i^n \left(softmax(W, x_i, h) - y_{ih}\right) x_i^T\right] \\
&= \left[\frac{1}{n}\sum_i^n \left(softmax(x_i W) - y_i\right) x_i^T\right]
\end{aligned}$$

Here we can constate that the gradient give us the features multiply by the error between what we predict ($softmax$) and the real values (y). If the error is big, the x_i will have more weight in the computation of the gradient.

So we can compute the optimal W thanks to a gradient descent:

$$\begin{aligned}
W^{i+1} &= W^i - \eta \nabla_W H(y, p(c|X)) \\
&= W^i - \eta \frac{1}{n}\sum_i^n (softmax(x_i W) - y_i) x_i^T
\end{aligned}$$